An Exact Comparison of Global, Partitioned, and Semi-Partitioned Fixed-Priority Real-Time Multiprocessor Schedulers

Artem Burmyakov
Faculty of Computer Science and Engineering
Innopolis University
Russia

Borislav Nikolić
Faculty of Information Technology
Belgrade Metropolitan University
Serbia

ABSTRACT

We evaluate the performance of global, partitioned, and semi-partitioned fixed-priority (FP) multiprocessor schedulers for sporadic real-time tasks. By conducting a range of exhaustive experiments, we support a prior observation of Baruah regarding the incomparability of global and partitioned fixed-priority schedulers, by demonstrating the existence of multiple task sets, which are schedulable by either one or another approach, but not both simultaneously. In addition, we demonstrate that a suboptimal semi-partitioning heuristic, named Deadline Monotonic with Priority Migration (DM-PM), in certain cases outperforms both global and partitioned FP schedulers. Unlike other works, our evaluation is based on an exact schedulability test for global scheduling, as well as relies on optimal task priorities for global FP, and an optimal partitioning of tasks to processors. However, due to the exponential computational complexity of the available exact schedulability test, our evaluation is constrained to a limited number of tasks in a set.

On average, partitioned scheduling performs better for a lower number of heavy tasks in a task set, while global scheduling is the opposite. The schedulability gain of one approach over another might exceed 50%, for both cases.

To make such an evaluation possible, we derive a method to compute optimal task priorities for global FP by using a set of pruning rules. We demonstrate that the gain of of such optimal priorities over other widely used priority assignment heuristics, including Deadline Monotonic (DM), can be significant, in certain cases. Moreover, we evaluate the performance of an optimal task partitioning over the available suboptimal partition heuristics, namely first-fit decreasing (FFD) and worst-fit decreasing (WFD), and confirm no significant performance difference between them, for considered task settings. Software tools used in our evaluation are implemented using C++ libraries, and are publicly available.

KEYWORDS

multiprocessor scheduling, fixed priorities, real-time systems, global and partitioned scheduling, optimal priorities

Authors' addresses: Artem Burmyakov, Faculty of Computer Science and Engineering Innopolis University, Russia; Borislav Nikolić, Faculty of Information Technology Belgrade Metropolitan University, Serbia.

1 INTRODUCTION

Real-time processor scheduling is widely used in the development of time-critical embedded systems, to allocate processors to pending jobs, by respecting certain deadlines. Nowadays, due to an increased computational demand of such systems, there is a trend towards using multiple processors, instead of a single one. However, multiprocessor scheduling is far less understood, as compared to a single processor scheduling.

To model the computational requirements of a real-time system, we employ the sporadic task model of Liu and Layland [31]. In this case, a system is represented as a set of tasks, wherein each task generates a potentially infinite sequence of jobs, with certain computational requirements and deadlines for completion. A scheduling algorithm is used to choose jobs to be executed first, with an aim to meet all jobs deadlines. In turn, a schedulability test is used to verify that no job can ever miss a deadline, under a given scheduling algorithm. We assume tasks to be scheduled by fixed-priority (FP) scheduling algorithm [3, 31], that operates under the assumption of each task having a static priority, and allocates processors to jobs of higher priority tasks first.

In our analysis we compare the performance of three main strategies for multiprocessor scheduling, namely global, partitioned, and semi-partitioned scheduling. While global scheduling allows jobs to freely migrate between processors (that is, job execution can be preempted upon one processor, and resumed upon a different one), partitioned scheduling statically maps each task to a certain processor, so that all jobs of that task are restricted to execute upon that processor only. In turn, semi-partitioned scheduling generalizes global and partitioned scheduling, by statically mapping the most of the tasks to certain processors, while allowing a few remaining tasks to freely migrate between processors.

Baruah [6] has previously illustrated with an example that global and partitioned schedulers are incomparable, meaning that there are task sets schedulable either by one or another strategy.[1] In this work we provide a missing thorough quantitative evaluation for the Baruah's observation, for fixed-priority scheduling, by conducting a range of exhaustive experiments, as well as extend his observation to semi-partitioned approaches. As opposed to all other available works, the evaluation that we provide is exact, rather than approximate, as in all other available works, as it relies on an exact schedulability test and an optimal priority assignment for tasks. However, due to the exponential computational complexity of the available exact test, our evaluation is constrained to a limited number of tasks in a set. In any case, our evaluation gives some glimpse

[1] Note that Baruah [6] considered the earliest deadline first (EDF) scheduler, not FP.

on the behavior of considered scheduling strategies based on an exact schedulability test.

All other available works comparing global and partitioned schedulers insist on a strong dominance of partitioned over global scheduling, in terms of the ratio of schedulable task sets [5, 21, 24, 29]. Such a result is however counterintuitive, that a higher freedom of executing jobs provided by a global scheduling results in a lower schedulability. In fact, due to having a reduced degree of freedom, partitioned scheduling is unable to schedule any task set, where the number of heavy tasks[2] exceeds the number of available processors[3]. On the other hand, global scheduling, according to our evaluation, succeeds to schedule many of such task sets; such a schedulability gain might exceed 50%, under certain settings.

Our skepticism about a strong dominance of partitioned over global scheduling mainly lies in the fact that the available works comparing these approaches are inaccurate, due to the following limitations:

(1) all these works rely on inexact schedulability tests for global schedulers, that have a high degree of pessimism in the provisioning of the demand for resources[4];
(2) these works use suboptimal task priority assignments for global FP, that significantly reduce its performance[5].

Theoretically, global scheduling generalizes partitioned scheduling. The fact that global FP scheduling fails to schedule some task sets, which are however schedulable by partitioned scheduling, only demonstrates the suboptimality of static task priorities over dynamic ones. On another side, from a practical perspective, global scheduling implies significant performance overheads related to task migrations between processors (e.g. due to cache affinity and memory bus contention, etc.), that can worsen the actual schedulability with respect to partitioned scheduling. Indeed, global scheduling is preferable only when the schedulability gain over partitioned scheduling is large, in order to compensate those practical drawbacks implied by global scheduling.

Semi-partitioned scheduling aims at combining the advantages of global and partitioned scheduling by allowing only a subset of tasks to migrate between processors, while keeping the remaining tasks statically fixed to some processors [2]. Theoretically, an optimal semi-partitioned scheduling should provide the same schedulability gain as global scheduling, at a lower number of task migrations. However, the schedulability analysis of semi-partitioned systems is even harder. Several semi-partitioned heuristics are available, such as Deadline Monotonic with Priority Migrations (DM-PM) by Kato and Yamasaki [28], which however impose strong restrictions on task priorities (e.g. tasks allowed to migrate have higher priorities than tasks statically assigned to processors), what leads in some cases to a lower schedulability gain than for global scheduling (see Section 3 for evaluation results).

[2]Task is said heavy, if its utilization exceeds 0.5

[3]If the number of heavy tasks exceeds the number of processors, then at least two heavy tasks should be assigned to the same processor. However, two heavy tasks are unschedulable upon a single processor, as their aggregated utilization exceeds 1.

[4]As shown by Burmyakov *et al.* [18], under certain settings, the most accurate sufficient schedulability tests fail to confirm the schedulability of more than half of task sets, that are in fact schedulable.

[5]There is no optimal task priority assignment known for global FP scheduling; thus, heuristics are used instead.

Motivated with the aforementioned reasoning, below we provide an exact comparison of schedulability performance for global, partitioned, and semi-partitioned DM-PM multiprocessor scheduling. For now, we limit the analysis to global FP scheduling (GFP), to be able to reuse an exact schedulability test of Burmyakov *et al.* [18]. We also use an optimal priority assignment for tasks, that is determined by an exhaustive search combined with some search-space pruning methods, that we derive. Finally, for a fair comparison, rather than reusing the available suboptimal heuristics for partitioned scheduling, we compute an optimal partitioning of tasks to processors, by using an exhaustive search.

Recently, in addition to schedulability performance, multiple other aspects have been addressed for the considered scheduling strategies, such as energy efficiency, temperature management, efficiency on heterogeneous platforms, which are however kept outside the scope of this work [22, 32–34, 37].

The paper is structured as follows. First, in Section 1.1, we introduce a system model and assumptions. Then, in Section 1.2, we provide an overview of related works, and later, in Section 1.3, we list the contributions of this paper. In Section 2, we recall an essential background on multiprocessor scheduling strategies. In particular, in Section 2.3.1 we provide details on the exact schedulability test for global FP, that is used in our work, and in Section 2.3.2 we derive a method to compute optimal task priorities, as well as we recall the available suboptimal heuristics for priority assignment. Finally, in Section 3, we report evaluation results, and in Section 4 we summarize our findings and conclude our study.

1.1 System Model and Assumptions

We model a real-time system by a set of sporadic tasks $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$, with each task $\tau_i = (C_i, D_i, P_i)$ characterized by a constant execution time C_i, a relative deadline D_i, and a minimum job interarrival time P_i. We consider constrained deadlines $D_i \leq P_i$. Each task $\tau_i \in \mathcal{T}$ generates a potentially infinite sequence of jobs, at arbitrary time instants, such that the time distance between any two consequitive job releases of τ_i is at least P_i time units. Each job of τ_i has a constant execution demand of C_i time units, and a relative deadline D_i counted from its release time. A task set utilization is defined by $U_{\mathcal{T}} = \sum_{i=1}^{n} C_i/P_i$, and an individual utilization of task τ_i is $u_i = C_i/P_i$. Task τ_i is said heavy, if $u_i > 1/2$.

Our assumptions are the following. Tasks are scheduled by FP scheduling upon m identical processors. Tasks in $\mathcal{T}$ are ordered by decreasing priorities. There are no dependencies between jobs, as well as no preemption nor migration costs. Task parameters C_i, D_i, P_i are integers, and scheduling decisions are taken at discrete time instants $\mathbb{N}_0 = 0, 1, 2, \ldots$ (tick-driven scheduling). We assume no preemption or migration overheads, as well as no release jitters for tasks.

Task set $\mathcal{T}$ is said schedulable, if no job can miss its deadline, for any legal scenario of job releases, for a given number of processors.

In this work we focus on fixed-priority (FP) scheduling, that operates under the assumption that tasks have distinctive static priorities, and that jobs of higher-priority tasks are executed first. In Figure 1, we depict an example of a global FP schedule (GFP) for sporadic tasks $\tau_1 = (2, 4)$, $\tau_2 = (3, 5)$, and $\tau_3 = (4, 6)$, with implicit

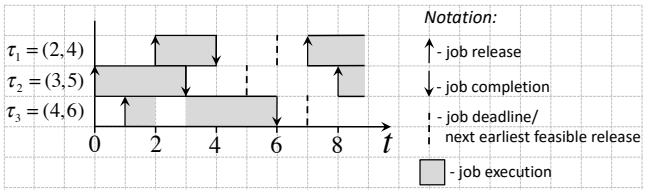


Figure 1: An example GFP schedule (illustrates the graphical notation used throughout this paper)

Table 1: System model notation

Symbol	Meaning
m	The number of available processors
$\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$	A set of n real-time sporadic tasks
$\tau_i = (C_i, D_i, P_i)$	A real-time sporadic task
C_i	An execution demand of a job of task τ_i
D_i	A relative deadline of a job of task τ_i
P_i (with $P_i \geq D_i$)	A minimum time separation between consecutive job releases by a sporadic task τ_i
$u_i = C_i/P_i$, $d_i = C_i/D_i$	Utilization and density of an individual task τ_i, respectively
$\mathcal{T}_j = \{\tau_{j_1}, \ldots, \tau_{j_k}\}$	A subset $\mathcal{T}_j \subset \mathcal{T}$ of k tasks assigned to the i-th processor, for partitioned scheduling
r_i	The worst-case response time for a job of task τ_i under fixed-priority scheduling (global, partitioned, or semi-partitioned)
$U_{\mathcal{T}} = \sum_{i=1}^{n} C_i/P_i$	An aggregated utilization of task set $\mathcal{T}$
$D_{\mathcal{T}} = \sum_{i=1}^{n} C_i/D_i$	An aggregated density of task set $\mathcal{T}$

deadlines $D_i = P_i$. Tasks are ordered by decreasing priorities. Tasks are scheduled upon $m = 2$ processors.

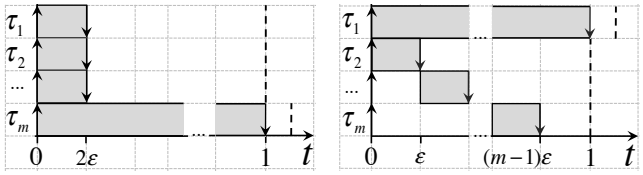
We reuse the graphical notation of Figure 1 throughout the paper. An upward arrow denotes a job release, and a downward arrow - a job completion. A dashed vertical line denotes a job deadline. Finally, a grey block denotes a job execution (equally, allocation of processor time).

Table 1 summarizes the system model notation used throughout the paper.

1.2 Related works

Baruah [6] was first to show the incomparability of global and partitioned schedulers, meaning that there are task sets schedulable either by global or partitioning approach. However, an exhaustive evaluation for this observation is still missing. Thus, we provide such an evaluation in this paper, considering FP scheduling, as well as we extend Baruah's observation to semi-partitioned DM-PM scheduling.

Below we list other related works analyzing the effectiveness of global, partitioned, and semi-partitioned scheduling strategies.



(a) Case 1: A set of m tasks with $U \to 0$ is unschedulable by GFP upon $m-1$ processors, with $m \to \infty$, for deadline-monotonic priority assignment

(b) Case 2: A different priority assignment allows to schedule the same task set upon 2 processors only, by GFP

Figure 2: The inefficiency of deadline-monotonic priority assignment for global FP scheduling

Dhall and Liu [21] were first to raise a doubt about the efficiency of global over partitioned scheduling. They showed that a task set with an arbitrary low utilization $U \to 0$ may require an arbitrary large number of processors $m \to \infty$, if scheduled by global FP, while it needs only 2 processors in case of partitioned FP. Their example is schematically depicted in Fig. 2. The task set is comprised of m periodic implicit-deadline tasks with a synchronous release of the first jobs. The $m-1$ higher priority tasks have arbitrary low execution times $C_i = \epsilon$, with $\epsilon \to 0$, and deadlines $D_i = 1$, for $i = 1, \ldots, m-1$. The lowest-priority task has $C_m = 1$ and $D_m = 1 + \epsilon/2$.

If scheduled by global FP, such a task set indeed requires m processors; if the number of processors is less, then task τ_m misses a deadline (see Figure 2a for $m-1$ processors). At the same time, partitioned FP requires only 2 processors: tasks $\tau_1, \ldots, \tau_{m-1}$ are partitioned to one processor[6], and task τ_m is partitioned to another processor.

However, the example of Dhall and Liu [21] in fact illustrates the inefficiency of deadline-monotonic priority assignment for global FP [7], rather than the inefficiency of gobal scheduling. Indeed, let us reassign task priorities, such that task τ_m with the largest period $P = 1 + \epsilon$ gets the highest priority, instead of the lowest as in the case of Figure 2a, and global FP succeeds to schedule such a task set upon 2 processors (see Figure 2b).

Below we list other relevant works to our study.

Lauzac *et al.* [29] compared the performance of global and partitioned rate-monotonic scheduling[8], restricting the analysis to periodic tasks with implicit deadlines, with a synchronous release of the first jobs at $t = 0$. In this case, they observed a very poor performance of global over partitioned scheduling.

Then, Baker [5] evaluated global earliest-deadline first (GEDF) scheduling over partitioned EDF [9], and observed a very poor performance of GEDF. His analysis however relied on sufficient schedulability tests, that significantly underestimate the performance of global schedulers. In fact, Biondi and Sun [13] showed later that available sufficient tests for global scheduling are so pessimistic,

[6] We assume that the total utilization of tasks $\tau_1, \ldots, \tau_{m-1}$ does not exceed 1, so that tasks are FP schedulable upon a single processor.
[7] For deadline-monotonic priority assignment, task with a lower deadline D_i gets a higher priority.
[8] For rate-monotonic scheduling, task with a lower period P_i gets a higher priority.
[9] EDF scheduler executes jobs with the earliest deadlines first.

that any task set, that is confirmed as schedulable by any of such tests, must be schedulable by a trivial partition heuristic as well.

Gracioli *et al.* [24] empirically evaluated global and partitioned EDF schedulers, by implementing them in a real-time operating system. Unlike earlier works, they considered preemption and migration overheads. They observed no significant difference in performance of global and partitioned scheduling. This result however relied on sufficient schedulability tests and suboptimal partition heuristics.

Finally, Brandenburg and Gül [15] empirically compared the performance of global and semi-partitioned schedulers. Unlike fully partitioned scheduling, a semi-partitioned scheduler allows some tasks to migrate between processors. Their analysis assumed tasks with implicit deadlines and an optimal global scheduler[10]. They concluded that global scheduling does not provide a significant schedulability gain, that justifies its implementation complexity and migration overheads that it causes.

1.3 Contributions of this paper

In this work we thoroughly evaluate the performance of global and partitioned FP schedulers, by comparing the achievable schedulability ratios for randomly generated sporadic task sets. To conduct an exact comparison for global and partitioned scheduling, unlike other works, we employ an exact schedulability test for GFP, optimal priority assignments for tasks, as well as an optimal partitioning of tasks to processors. We demonstrate that global and partitioned schedulers are incomparable: there are numerous task sets schedulable by either global or partitioned scheduler, but not both at the same time.

To make our exact evaluation possible, we derive a method to compute optimal task priorities for GFP, by employing search space pruning. We then demonstrate a significant gain of optimal task priorities over deadline-monotonic priorities, that are widely considered in the literature. We evaluate the performance of other suboptimal heuristics for priority assignment as well, namely DC, DCMPO, and DkC [19].

Finally, we conduct the performance evaluation of an optimal tasks partitioning over available suboptimal partition heuristics, namely first-fit decreasing (FFD) and worst-fit decreasing (WFD). We demonstrate no significant performance gain of an optimal partitioning over FFD and WFD heuristics, for considered experiment settings. For completeness, we also evaluate a semi-partitioned scheduling heuristic, DM-PM, which provides some schedulability gain over partitioned scheduling.

To the best of our knowledge, no results of such an evaluation has been yet published.

2 BACKGROUND ON FIXED-PRIORITY SCHEDULING

We first recall an essential background on fixed-priority scheduling, that we later use in our study.

[10] A scheduler is optimal, if it always finds a schedule with no deadlines missed, whenever such a schedule exists. An example of an optimal scheduler is provided by Baruah *et al.* [10].

2.1 Partitioned fixed-priority scheduling

For partitioned scheduling, each task from task set $\mathcal{T}$ is statically mapped to some processor. All jobs of a certain task are restricted to execute upon one specific processor only; no job migration between processors is allowed. To schedule tasks assigned to the same processor, we use deadline-monotonic (DM) uniprocessor scheduler[11], that is optimal among fixed-priority uniprocessor schedulers[12] for sporadic tasks with constrained deadlines [30].

To test the DM schedulability of tasks assigned to the same processor, we employ the following exact test of Joseph and Pandya [27]. Let tasks $\tau_{i_1}, \ldots, \tau_{i_k} \in \mathcal{T}$ be assigned to the same processor (tasks are assumed to be sorted by decreasing priorities). Assuming that tasks $\tau_{i_1}, \ldots, \tau_{i_{k-1}}$ are confirmed schedulable, the lowest-priority task τ_{i_k} is DM schedulable, if and only if

$$r_{i_k} \leq D_{i_k}, \tag{1}$$

where r_{i_k} denotes the worst-case response time[13] for a job of τ_{i_k}, that equals to the smallest positive value satisfying the recursive equation

$$r_{i_k} = C_{i_k} + \sum_{\ell=1}^{k-1} \left\lceil \frac{r_{i_k}}{P_{i_\ell}} \right\rceil C_{i_\ell}.$$

The equation above is solved by using fixed-point search [16].

An open problem is an optimal partition algorithm of tasks to processors, that succeeds to find a schedulable partition whenever it exists. The problem of an optimal partitioning is known to be NP-hard in the strong sense, being a specific case of a well-known bin-packing problem [7, 26]. Consequently, a set of suboptimal partition heuristics has been introduced as well, including the first-fit decreasing (FFD) and the worst-fit decreasing (WFD) [9, 21]. In our work we evaluate the performance of an optimal partitioning together with FFD and WFD partition heuristics. To determine optimal partitions, we implement the ILP-based approach of Baruah and Bini [7] in CPLEX, which turns out to be significantly faster than a naive brute-force enumeration.

We next briefly recall the idea behind FFD and WFD heuristics. Assume that tasks $\tau_1, \ldots, \tau_k$, that are to be partitioned, are ordered by decreasing utilizations $u_i = C_i/P_i$. Let processors be numbered by $1, \ldots, m$. FFD and WFD are recursive procedures, that work as follows. Let $\mathcal{T}_j$ denote tasks assigned to the j-th processor by a current iteration. For FFD, after proceeding with tasks $\tau_1, \ldots, \tau_{i-1}$, task τ_i is assigned to processor j, that satisfies the following conditions:

- Task set $\mathcal{T}_j \cup \{\tau_i\}$ is schedulable by DM, meaning that condition (1) holds;
- For any processor $k < j$, task set $\mathcal{T}_k \cup \{\tau_i\}$ is unschedulable by DM.

Instead, WFD assigns task τ_i to processor j, that satisfies the following conditions:

- Task set $\mathcal{T}_j \cup \{\tau_i\}$ is schedulable by DM;
- The aggregated utilization $U_{\mathcal{T}_j} + u_i$ is minimal, as compared to any other processor, that is able to accommodate task τ_i.

[11] Deadline-monotonic scheduler assigns higher priorities to tasks with lower deadlines D_i.

[12] If a task set is unschedulable by DM scheduler, then it is unschedulable by any other fixed-priority uniprocessor scheduler.

[13] The response time of a job is the time between its release and completion.

Regardless of the selected partitioning strategy, partitioned scheduling is unable to schedule any task set, that has the number of heavy tasks exceeding the number of processors. Indeed, in this case at least two heavy tasks should be assigned to the same processor, and as their aggregated utilization exceeds 1, they are unschedulable upon a single processor. Semi-partitioned or global scheduling should be used for such task sets instead, that is described next.

2.2 Semi-partitioned fixed priority scheduling

Semi-partitioned scheduling, unlike partitioned scheduling, allows a few tasks to migrate across processors, while other tasks are statically fixed to processors [1, 2]. For our evaluation we employ a semi-partitioned scheduling heuristic by Kato and Yamasaki [28], Deadline Monotonic with Priority Migration (DM-PM). First, tasks are assigned to processors following some partition heuristic. We evaluated several partition heuristics and confirmed FFD to provide the best schedulability gain (see Section 2.1). Then, those tasks, which cannot be assigned to any processor without violating system schedulability, are splitted across several processors. All tasks assigned to the same processor, including fractions of splitted tasks, are scheduled by uniprocessor FP scheduler, with an additional requirement of splitted tasks to have the highest priorities[14]. Semi-partitioned scheduling based on FFD heuristic must always outperform FFD partitioned scheduling. However, due to the requirement of the highest priorities to splitted tasks, under certain settings, semi-partitioned scheduling performs worse than an optimal fixed-priority partitioned scheduling (see evaluation details in Section 3).

2.3 Global fixed-priority scheduling

Global scheduling allows jobs to migrate between processors, unlike partitioned scheduling [21]. Thanks to such migrations allowed, global fixed-priority (GFP) scheduling succeeds to schedule many task sets with a large number of heavy tasks, which otherwise are unschedulable by partitioning (see a thorough evaluation of this statement later in Section 3). On the opposite side, GFP fails to schedule many task sets with a lower number of heavy tasks, which in turn are schedulable by the partitioned scheduling. Moreover, global scheduling imposes additional practical overheads, such as job migration cost.

We note that, theoretically, global scheduling generalizes partitioned scheduling. Whenever partitioned FP outperforms global FP, it only demonstrates the suboptimality of static task priorities over dynamic ones for global scheduling, rather than the suboptimality of global scheduling itself. Despite its importance, global scheduling is far less understood, as compared to uniprocessor scheduling. A key problem is that, unlike uniprocessor scheduling, no worst-case execution scenario is known for global scheduling, that could allow to derive a fast exact schedulability test. In fact, the schedulability analysis for global scheduling is an NP-hard computational problem in the strong sense. All available exact tests suffer from a high computation time and space complexity, that makes them intractable for realistic task sets [14, 23, 35].

To overcome the lack of an efficient exact schedulability test, several sufficient tests have been derived instead [4, 6, 11, 25], that are significantly faster compared to exact ones, but very pessimistic. According to a recent evaluation of Burmyakov *et al.* [18], such a pessimism exceeds 50%, for certain task settings, meaning that a sufficient test fails to confirm the schedulability of more than half of task sets, that are in fact schedulable, according to an exact test.

Sun and Di Natale [36] further supported our observation, by demonstrating that the pessimism of the available sufficient tests is so significant, that any task set that is confirmed schedulable by any existing GFP sufficient test, can be in fact schedulable by a trivial partitioning heuristic. Later, Brandenburg and Gul [15], and Biondi and Sun [13] provided an experimental evidence for this thesis.

To conduct our evaluation, we employ an exact test of Burmyakov *et al.* [18], that is significantly faster compared to all other exact tests, and remains tractable for systems with a larger number of tasks. The details on this exact test are provided later in Section 2.3.1. The available sufficient tests are excluded from our study, due to their known pessimism. Moreover, to perform a fair evaluation of global FP scheduling, we use optimal task priorities[15] instead of the available suboptimal heuristics for assigning task priorities. In order to determine optimal task priorities, we use an exhaustive search combined with some search space pruning methods, that are described later in Section 2.3.2.

We note that no performance comparison of optimal and suboptimal priority assignments, such as the one proposed by Davis and Burns [20], has been yet conducted. We aim to perform such an analysis in the future work.

For the sake of completeness, we next provide a brief overview of an exact schedulability test of Burmyakov *et al.* [18], that is used in our evaluation. After that, in Section 2.3.2, we derive a method to compute optimal task priorities for GFP. Finally, in Section 2.3.3, we review the available heuristics for suboptimal priority assignment for tasks, that we later evaluate in Section 3 against an optimal priority assignment.

2.3.1 An exact schedulability test for GFP *by using state space pruning.* Bonifaci and Marchetti-Spaccamela [14] proposed to test the GFP schedulability of a set of sporadic tasks $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$ by traversing a finite state transition graph, which represents all feasible system states during its execution. Each state in such a graph is modeled by a tuple

$$\{(c_i,\ d_i,\ p_i)\}_{i=1}^{n},$$

with each triple $(c_i,\ d_i,\ p_i)$ corresponding to task $\tau_i = (C_i, D_i, P_i)$. Parameter $c_i \in [0, \ldots, C_i]$ denotes the remaining execution time of a pending job of τ_i (if it has any), parameter $d_i \in [0, \ldots, D_i]$ denotes time left until the deadline of that job, and finally p_i denotes time left until τ_i is allowed to release the next job.

The initial state in a graph is $\{(c_i = 0,\ d_i = 0,\ p_i = 0)\}_{i=1}^{n}$, when no job has been yet released. The transition between states corresponds to one clock tick in an execution schedule. The successors of a given state have task parameters updated respectively, for the transition of one clock tick, as well as represent all feasible release scenarios for new jobs.

We provide an example of such a graph in Fig. 3b, for tasks $\mathcal{T} = \{\tau_1 = (2, 5), \tau_2 = (3, 7)\}$ with implicit deadlines $D_i = P_i$, scheduled upon $m = 1$ processor. Please note that we choose so

[14] This requirement is imposed to simplify the schedulability analysis of a system

[15] A priority assignment is said optimal, if it fails to schedule a given task set only when indeed no other feasible priority assignment exists, that makes a task set schedulable.

tiny task parameters and $m = 1$ processor in order to be able to depict an entire state transion graph for our example; for larger task parameters, the graph size increases drastically, so that we are unable to depict it, due to space limitations.

To test the schedulability of a real-time system, Bonifaci and Marchetti-Spaccamela [14] traverse its respective state transition graph, until they either find a state with a violated deadline, having $c_i > d_i$ for some τ_i, or all graph states are confirmed schedulable. Such a test is proved to terminate always, due to the finity of a state transition graph for any arbitrary system.

The test of Bonifaci and Marchetti-Spaccamela significantly outperformed all other state-of-the-art exact tests in terms of computation time. However, its computational complexity remained exponential, making the test intractable for systems with a larger number of tasks.

Later, Burmyakov *et al.* [18] have significantly speeded up Bonifaci's test by employing the idea of a state space pruning, that is as follows. The aim of a graph traversal is to check the existence of a state with a missed deadline. Thus, Burmyakov's test aims to reduce the number of graph states to be examined by introducing a set of safe pruning conditions, that allow to exclude all those graph branches, that cannot lead to a state with a missed deadline. One of such pruning conditions is the necessary condition for a deadline miss (see Eq. (19) in [18]): if some graph state violates such a necessary condition, then none of its successors has a missed deadline, and thus, there is no need to traverse those states. In Fig. 3, we illustrate the application of such a pruning condition graphically; states violating a necessary unschedulability condition are crossed, and all states pruned from the analysis are hatched.

Thanks to a range of pruning conditions derived, Burmyakov's exact test remains tractable for larger task sets.[16]. Therefore, we employ this test for our analysis.

2.3.2 Search for optimal task priorities for GFP. Next we describe our computation of optimal task priorities for global FP scheduling, that we use in our evaluation.

For n tasks, there are $n!$ feasible priority assignments to be checked. The number of cases can be however significantly reduced, by exploiting the following observation.

Let us first discuss this observation with an illustrative example. Consider a schedule depicted in Fig. 4a, for implicit-deadline sporadic tasks $\tau_1 = (2, 3)$, $\tau_2 = (3, 5)$, and $\tau_3 = (4, 8)$, that are scheduled upon $m = 2$ processors. These tasks are sorted by decreasing priorities. Let us now swap the priorities for tasks τ_1 and τ_2, so that τ_2 gets the highest priority, as depicted in Fig. 4b. Such a change of priorities for τ_1 and τ_2 does not affect the execution of τ_3, because for $m = 2$ processors tasks τ_1 and τ_2 will always execute their jobs without any interference, regardless of their priority order.

We next generalize such an observation, by showing that the priority order for the m highest priority tasks (m equals the number of processors) is irrelevant for the schedulability of $\mathcal{T}$, as any job of these tasks cannot be interferred at any time.

Observation 1 (The schedulability invariance with respect to priority order among m highest priority tasks). *Let $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$ be scheduled upon m processors, with tasks in $\mathcal{T}$ sorted by decreasing priorities. Let $\mathcal{T}$ be transformed into $\mathcal{T}' = \{\tau_{i_1}, \ldots, \tau_{i_m}, \tau_{m+1}, \ldots, \tau_n\}$, by arbitrary permutting the priorities for the m highest priority tasks $\tau_1, \ldots, \tau_m$ (that is, for such $\mathcal{T}'$, it holds that $i_\ell \in \{1, \ldots, m\}$, for each $\ell = 1, \ldots, m$); the priorities for $\tau_{m+1}, \ldots, \tau_n$ remain unchanged. It follows that such a change of priorities does not affect the schedulability of any of tasks $\tau_{m+1}, \ldots, \tau_n$. A transformed $\mathcal{T}'$ is schedulable if and only if $\mathcal{T}$ is schedulable.*

Such a change of priorities among the m highest priority tasks $\tau_1, \ldots, \tau_m$ does not affect their schedulability, because they anyway get a processor as soon as they release jobs, and they never suffer any interference. Interestingly, this change of priorities has no effect on τ_{m+1} to τ_n either, because the response times for higher priority tasks $\tau_1, \ldots, \tau_m$ remain unchanged, and thus the resulted worst-case interference on any of tasks $\tau_{m+1}, \ldots, \tau_n$ remains unchanged as well.

According to Observation 1, when searching for an optimal priority assignment for tasks, to reduce the number of cases to be examined, we simply discard all but one of combinations of priority assignments that satisfy Observation 1, from the analysis. For example, for three tasks τ_1, τ_2, τ_3, that are scheduled upon $m = 2$ processors, it is sufficient to check only these priority assignments: $\{\tau_1, \tau_2, \tau_3\}$, $\{\tau_2, \tau_3, \tau_1\}$, $\{\tau_3, \tau_1, \tau_2\}$. In a general case, the number of cases to be checked is reduced from $n!$ to $n!/m!$.

Later in Section 3 we demonstrate a significant gain of optimal task priorities over some widely used suboptimal priority assignment heuristics, which are listed next. Such a gain might exceed $60 - 90\%$ for certain settings. We also evaluate an optimal priority assignment against other suboptimal heuristics.

2.3.3 Priority assignment heuristics. Finding an optimal priority assignment for tasks has an exponential complexity for the number of tasks n. However, a set of simpler suboptimal heuristics for priority assignment is available as well, namely DCMPO, DkC, and *DC*, that assign task priorities as follows [19]:

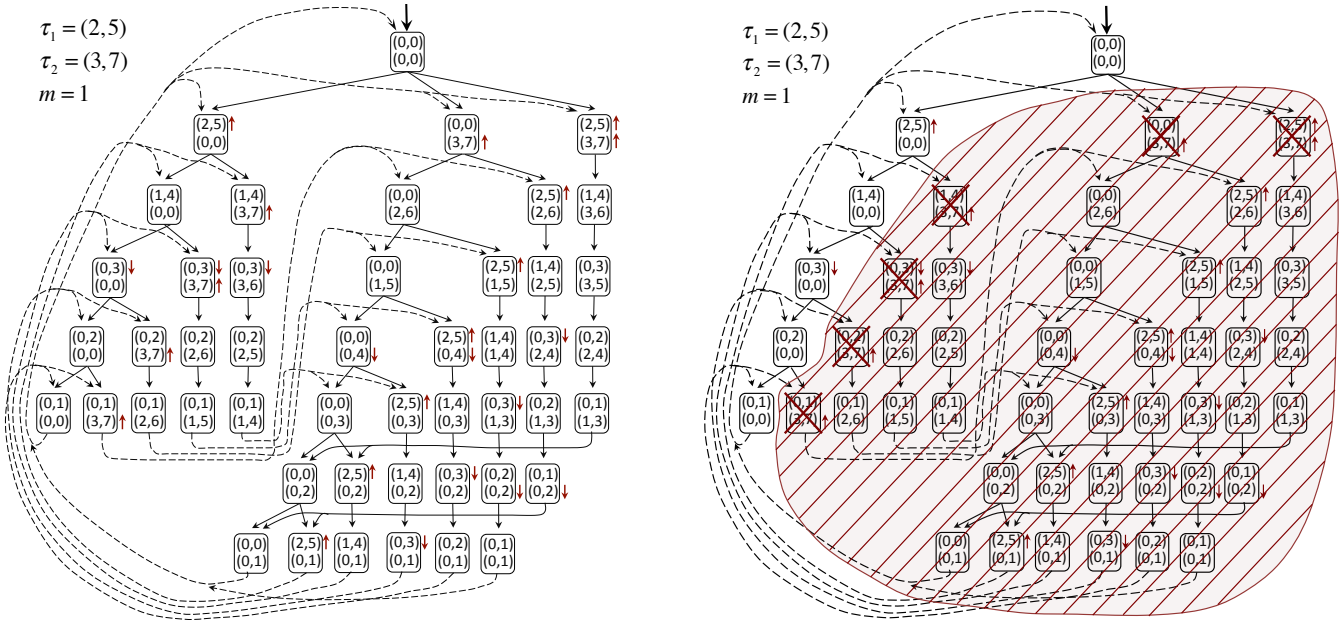
(1) DCMPO assigns a higher priority to task τ_i with a lower value of $(D_i - C_i)$;
(2) DkC assigns a higher to task τ_i with a lower value of $(D_i - k \times C_i)$, where k is computed by
$$k = \frac{m - 1 + \sqrt{5m^2 - 6m + 1}}{2m}$$
(3) DC assigns a higher priority to a task with a higher value of C_i/D_i

In Section 3 we evaluate the performance of these suboptimal heuristics against an optimal priority assignment.

3 EVALUATION

We finally report the evaluation results for global, partitioned, and semi-partitioned fixed-priority schedulers. First, in Section 3.1, we describe an algorithm used to generate random task sets for our evaluation. Then, in Section 3.2, we evaluate the schedulability performance of global, partitioned, and semi-partitioned multiprocessor schedulers for various settings. After that, in Section 3.3, we evaluate the schedulability gain of an optimal priority assignment of GFP over available suboptimal heuristics, namely DCMPO, DkC, DC, and DM. Then, in Section 3.4, we evaluate the schedulability gain of an optimal task partitioning over suboptimal FFD and WFD

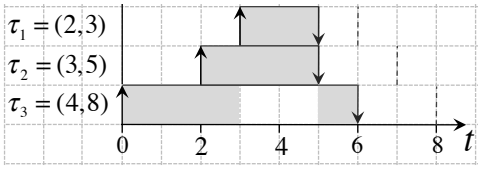
[16] A detailed performance evaluation is available [17]



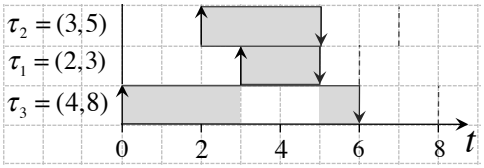
(a) State transition graph by Bonifaci and Marchetti-Spaccamela [14] (breadth-first exploration)

(b) State transition graph pruned by a necessary unschedulability condition (19) from [18]

Figure 3: A graphical illustration of a state space pruning by Burmyakov *et al.* [18]



(a) Case 1: τ_1 has the highest priority



(b) Case 2: τ_2 has the highest priority

Figure 4: Swapping the priorities for tasks τ_1 and τ_2 does not affect the schedulability of τ_3. Tasks are scheduled upon $m = 2$ processors

partition heuristics. Finally, for completeness, in Section 3.5 we report the runtime of an exact schedulability test for the examined GFP scheduling policies. Software tools used in our evaluation are implemented in C++ programming language, and are publicly available[17].

[17] https://sites.google.com/site/artemburmyakov/home/papers

3.1 Task set generation

We generate random sporadic task sets $\mathcal{T} = \{\tau_i = (C_i, P_i)\}$ by specifying the following input parameters:

(1) the number of tasks n;
(2) the number of heavy tasks[18] n_{heavy};
(3) the total task set utilization $U_{\mathcal{T}}$;
(4) the ratio between task periods and deadlines P_i/D_i;
(5) the range for task deadlines;
(6) the ratio between the maximum and minimum task deadlines $D_{\max}/D_{\min}$.

We randomly generate task sets for the settings reported in Table 2. In each experiment, one key parameter is varied, while the rest are left equal to the default values in Table 2. For each value of a varying parameter, we randomly generate at least 60 task sets. The choice for parameter values is constrained by the runtime of an exact schedulability test of Burmyakov *et al.* [18] used in our evaluation.

Due to the runtime limitations of an exact schedulability test, we restrict task parameters C_i, D_i, and P_i to small integers. This restriction makes it impossible to reuse the available generators for task sets, such as the ones derived by Bini and Buttazo [12] and Davis and Burns [19], because they are not applicable for small integer values for C_i, D_i, and P_i. Thus, below we propose another method to generate task sets, by employing integer-linear programming (ILP).

[18] For the experiments with a varying ratio of P_i/D_i, we redefine a heavy task as the one having its density $C_i/D_i > 0.5$, otherwise, in case $P_i/D_i > 2$, we are unable to generate any heavy task in a set.

Table 2: Key parameters: default values

Settings		
Number of processors, m	2	4
Number of tasks, n	7	14
Number of heavy tasks, n_{heavy}	2	4
Task set utilization, $U_{\mathcal{T}}$ (excl. exps. for varying P_i/D_i ratio)	1.75	3
Task set density, $D_{\mathcal{T}}$ (for exps. with varying P_i/D_i ratio)	2	4
Ratio between the maximum and minimum task deadlines, $D_{\max}/D_{\min}$	20	5
Range for task deadlines, D_i	[5; 120]	[7; 35]
Ratio between task periods and deadlines, P_i/D_i	1	1

We first randomly pick task deadlines D_i from the range $[D_{\min}, D_{\max}]$, such that a specified ratio of $D_{\max}/D_{\min}$ holds. Then, we compute task periods P_i for a given P_i/D_i ratio, and finally, execution times C_i, by solving the following ILP problem in CPLEX:

$$\begin{aligned} &\text{minimize } \left| U_{\mathcal{T}} - \sum_{i=1}^{n} C_i/D_i \right| \\ &\text{subject to} \\ &\quad 0 < C_i < D_i, \qquad i = 1, \ldots, n \\ &\quad \left| U_{\mathcal{T}} - \sum_{i=1}^{n} C_i/D_i \right| \leq \delta_{U_{\mathcal{T}}} U_{\mathcal{T}} \\ &\quad C_i/D_i \leq C_{i^*}/D_{i^*}, \qquad i = 1, \ldots, n, \end{aligned}$$

where C_i, $i = 1, \ldots, n$, are integer optimization variables, and $|x|$ denotes the absolute value of x. If the optimization constraints above are infeasible, then another set of task deadlines D_i is generated, and the process is repeated. To speed up computation, we allow a relative deviation $\delta_{U_{\mathcal{T}}} = 1.5\%$ between the specified value U and a cumulative utilization of tasks $\sum_{i=1}^{n} C_i/P_i$.

3.2 Experiments: the performance comparison of global and partitioned FP schedulers

We first compare the schedulability ratios for GFP, Semi-PFP, and PFP schedulers. The results are reported in Figs. 5a-14a, for a varying number of tasks n, heavy tasks n_{heavy}, the ratio $D_{\max}/D_{\min}$ of maximum to minimum task deadlines in a task set, the aggregated task set density $D_{\mathcal{T}}$, and the varying ratio P_i/D_i of task deadlines to periods, respectively. Note that for the experiments with implicit (that is $P_i/D_i = 1$) and explicit (that is $P_i/D_i > 1$) task deadlines we set different default values for $D_{\mathcal{T}}$.

For the experiments with $m = 2$ processors, we are able to provide the schedulability performance of GFP with optimal task priorities, denoted by GFP-OPT, as well as an optimal partitioning of tasks to processors for PFP scheduling, denoted by PFP-OPT. Optimal task priorities and partitioning are computed according to Sections 2.3 and 2.1. For the experiments with $m = 4$ processors and $n = 14$ tasks, due to a high computation time, instead of GFP-OPT, we report the results for GFP-DCMPO, DkC, and DC scheduling. We observe the incomparability of global, partitioned, and semi-partitioned schedulers; the schedulability gain of either one or another approach exceeds 40%, in certain cases. In particular, GFP significantly outperforms PFP for task sets with a larger number of heavy tasks, and task sets with larger ratios of the maximum to the minimum task periods. We also observe that multiple task sets are schedulable either by GFP, or Semi-PFP, or PFP, but not all schedulers simultaneously. Note that, unlike GFP and Semi-PFP schedulers, PFP is unable to schedule any task set with the number of heavy tasks exceeding the number of available processors, in case task deadlines equal to task periods, $D_i = P_i$.

3.3 Experiments: the schedulability gain of optimal task priorities for GFP

We next evaluate the schedulability gain of an optimal priority assignment[19] for GFP over the available suboptimal heuristics, which are listed in Section 2.3.3. The results are reported for $m = 2$ processors in Figs. 5b-14b. We observe that DkC and DCMPO priority assignment heuristics significantly outperform DC and DM, but still fail to schedule $5 - 20\%$ of task sets, that are schedulable by GFP-OPT. We also observe a poor performance of DM priority assignment, that fails to schedule $40 - 70\%$ of task sets, that are schedulable by GFP-OPT.

3.4 Experiments: the schedulability gain of an optimal task partitioning

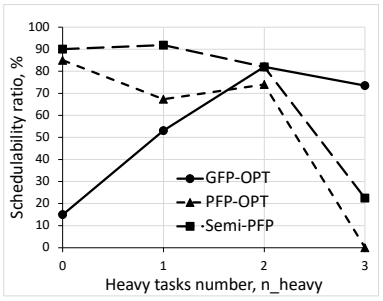
We also evaluate the performance of suboptimal FP partition heuristics, namely the first-fit decreasing (PFP-FFD) and the worst-fit decreasing (PFP-WFD), over an optimal partitioning (PFP-OPT) determined by a brute-force search, according to Section 2.1. The results are reported in Figs. 5-14, for $m = 2$ processors. In an average case, we observe no significant performance difference between an optimal and suboptimal partition heuristics.

3.5 Experiments: the runtime evaluation of an exact schedulability test

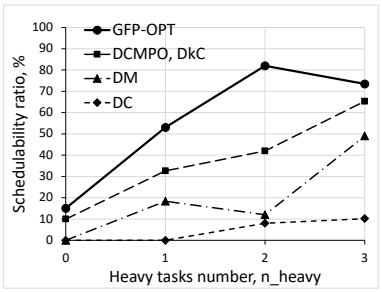
Finally, for completeness, we evaluate the computation time for the employed exact schedulability test. The results are reported in Figs. 5f-9f and 10e-14e. We observe a strong dependency of the test runtime on a chosen priority assignment heuristic. In particular, the runtime for DC is significantly lower than for other heuristics, but the resulted schedulability ratio is poor. On another side, the runtime for DM is large (unlike other heuristics, we could not conduct DM evaluation for $m = 4$, as it took too much time), but the schedulability ratio is lower than for DCMPO and DkC. The runtime for DCMPO and DkC is approximately the same. Note that the runtime for GFP-OPT is computed only for cases, when suboptimal priority assignment heuristics result in a deadline miss.

We also evaluate the number of examined priority assignments for GFP-OPT, by considering the pruning heuristics introduced in

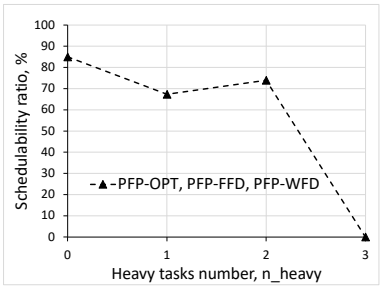
[19] Computed by an exhaustive search combined with pruning heuristics described in Section 2.3



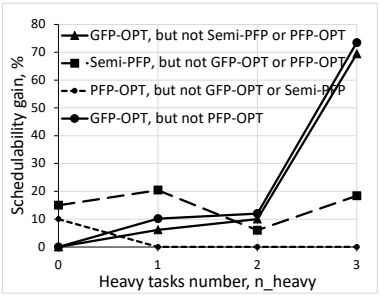
(a) Schedulability ratios for GFP, Semi-PFP, **and** PFP **schedulers**



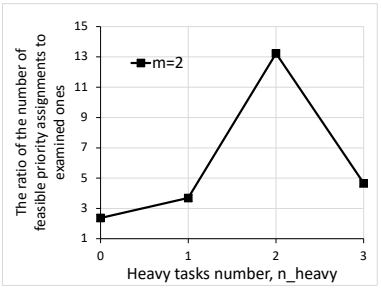
(b) Schedulability ratios of priority assignment heuristics for GFP



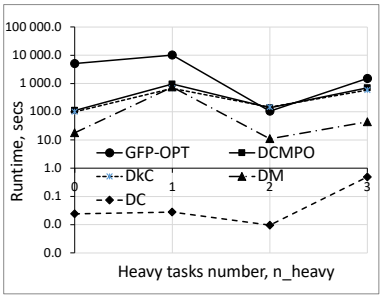
(c) Schedulability ratios of partitioning heuristics



(d) Schedulability gain of GFP, Semi-PFP, **and** PFP

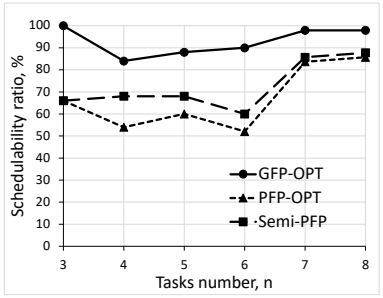


(e) Reduction of priority assignments to be examined for optimal GFP

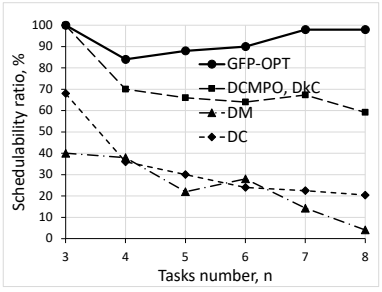


(f) Runtime of the exact schedulability tests for GFP **(logarithmic scale)**

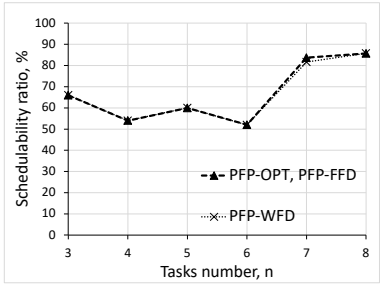
Figure 5: Evaluation results for a varying number of heavy tasks, n_{heavy}, for $m = 2$ processors



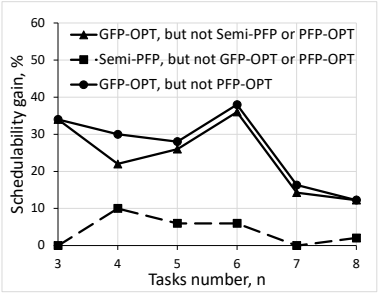
(a) Schedulability ratios for GFP, Semi-PFP, **and** PFP **schedulers**



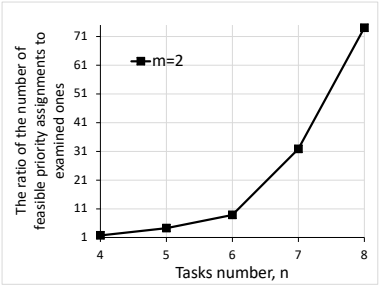
(b) Schedulability ratios of priority assignment heuristics for GFP



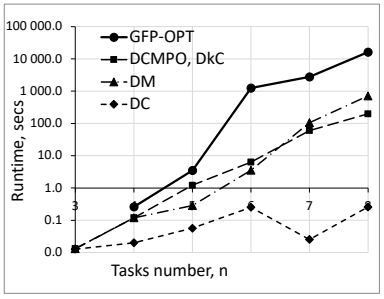
(c) Schedulability ratios of partitioning heuristics



(d) Schedulability gain of GFP, Semi-PFP, **and** PFP

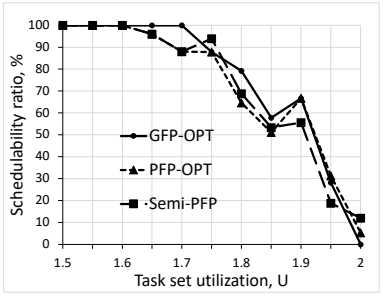


(e) Reduction of priority assignments to be examined for optimal GFP

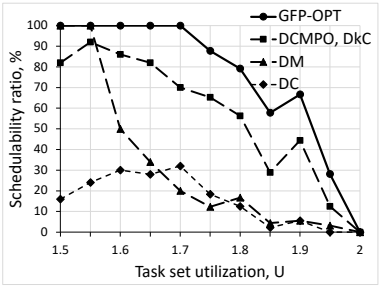


(f) Runtime of the exact schedulability tests for GFP **(logarithmic scale)**

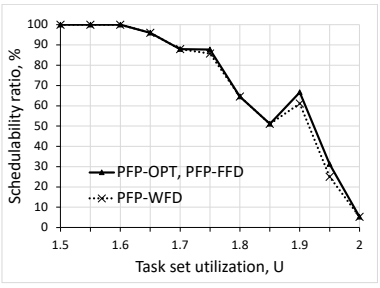
Figure 6: Evaluation results for a varying number of tasks n, for $m = 2$ processors



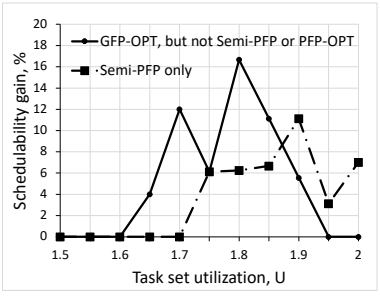
(a) **Schedulability ratios for** GFP, Semi-PFP, **and** PFP **schedulers**



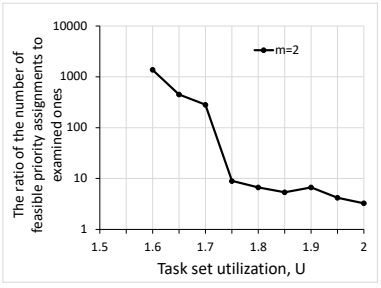
(b) **Schedulability ratios of priority assignment heuristics for** GFP



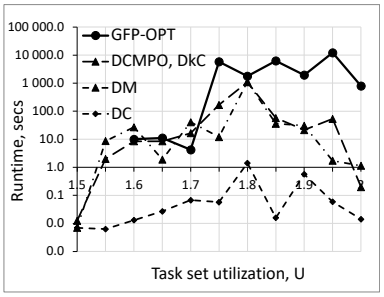
(c) **Schedulability ratios of partitioning heuristics**



(d) **Schedulability gain of** GFP, Semi-PFP, **and** PFP

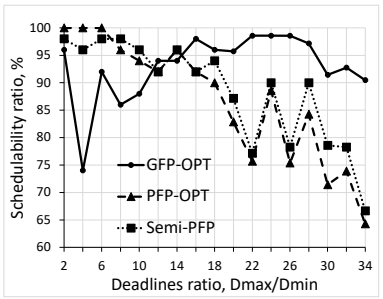


(e) **Reduction of priority assignments to be examined for optimal** GFP

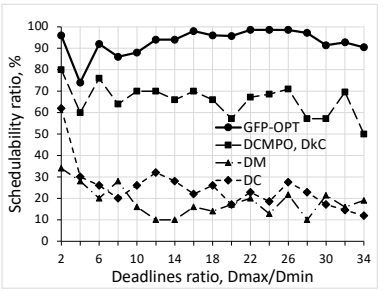


(f) **Runtime of the exact schedulability tests for** GFP **(logarithmic scale)**

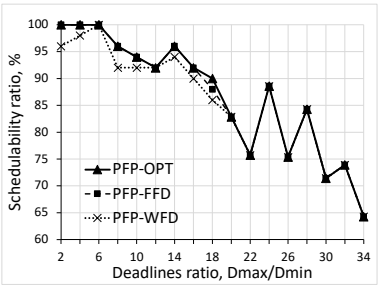
Figure 7: Evaluation results for a varying task set utilization $U_{\mathcal{T}}$, for $m = 2$ processors



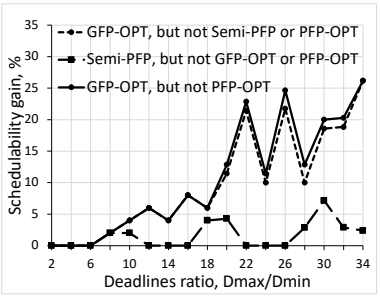
(a) **Schedulability ratios for** GFP, Semi-PFP, **and** PFP **schedulers**



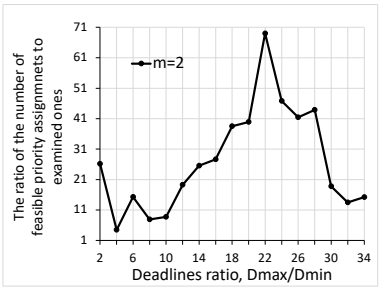
(b) **Schedulability ratios of priority assignment heuristics for** GFP



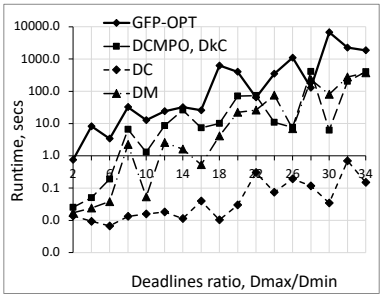
(c) **Schedulability ratios of partitioning heuristics**



(d) **Schedulability gain of** GFP, Semi-PFP, **and** PFP

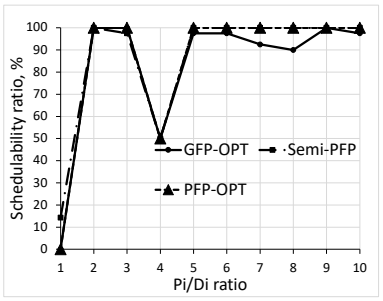


(e) **Reduction of priority assignments to be examined for optimal** GFP

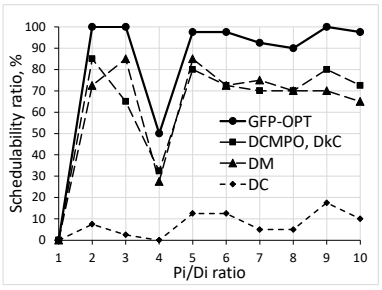


(f) **Runtime of the exact schedulability tests for** GFP **(logarithmic scale)**

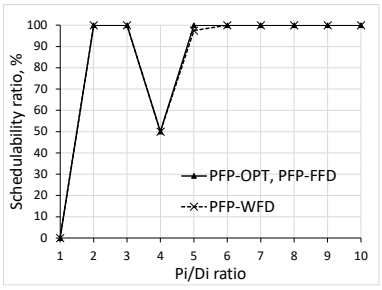
Figure 8: Evaluation results for a varying $D_{\max}/D_{\min}$ ratio, for $m = 2$ processors



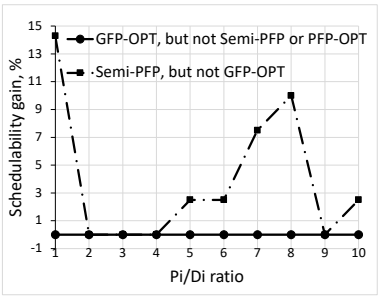
(a) **Schedulability ratios for** GFP, Semi-PFP, **and** PFP **schedulers**



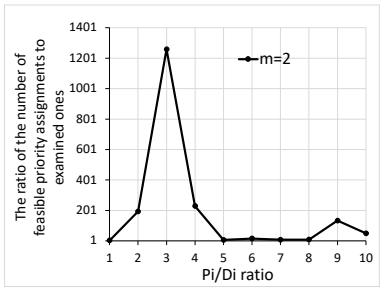
(b) **Schedulability ratios of priority assignment heuristics for** GFP



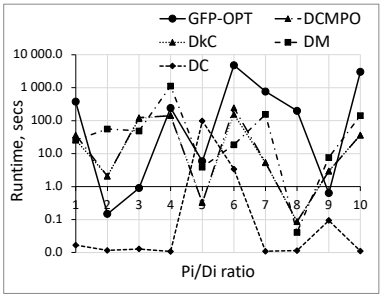
(c) **Schedulability ratios of partitioning heuristics**



(d) **Schedulability gain of** GFP, Semi-PFP, **and** PFP

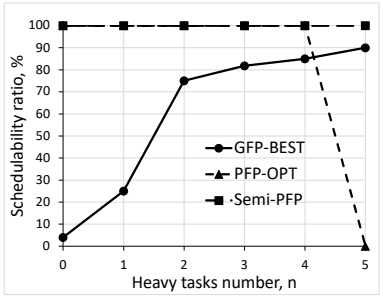


(e) **Reduction of priority assignments to be examined for optimal** GFP

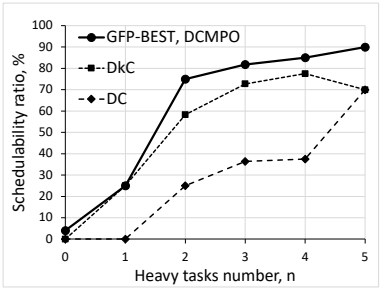


(f) **Runtime of the exact schedulability tests for** GFP **(logarithmic scale)**

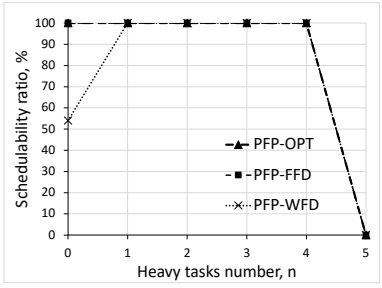
Figure 9: Evaluation results for a varying ratio of task periods to deadlines P_i/D_i, for $m = 2$ processors



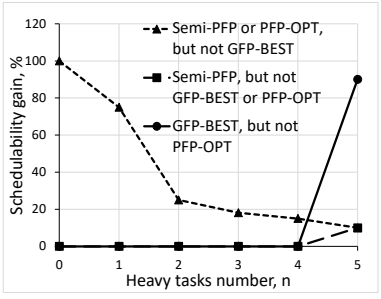
(a) **Schedulability ratios for** GFP, Semi-PFP, **and** PFP **schedulers**



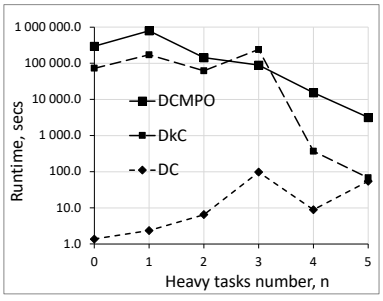
(b) **Schedulability ratios of priority assignment heuristics for** GFP



(c) **Schedulability ratios of partitioning heuristics**

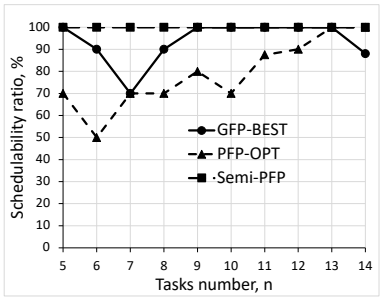


(d) **Schedulability gain of** GFP, Semi-PFP, **and** PFP

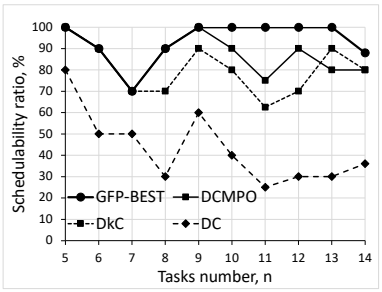


(e) **Runtime of exact** GFP **schedulability tests**

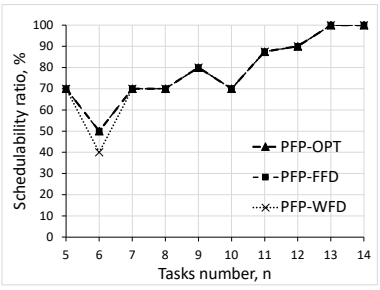
Figure 10: Evaluation results for a varying number of heavy tasks n_{heavy}, for $m = 4$ processors



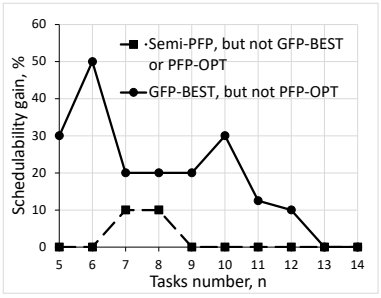
(a) **Schedulability ratios for** GFP, Semi-PFP, **and** PFP **schedulers**



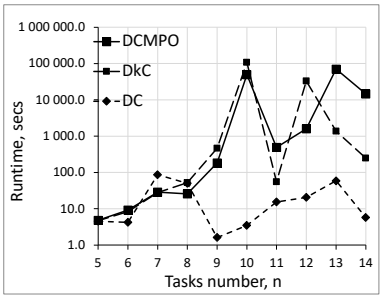
(b) **Schedulability ratios of priority assignment heuristics for** GFP



(c) **Schedulability ratios of partitioning heuristics**

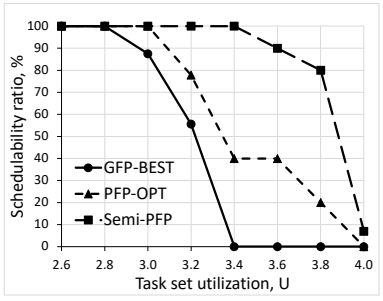


(d) **Schedulability gain of** GFP, Semi-PFP, **and** PFP

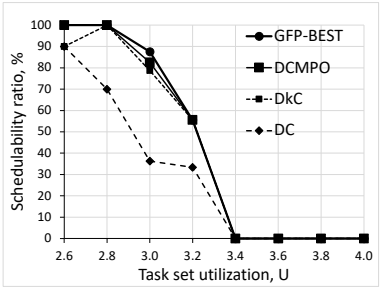


(e) **Runtime of exact** GFP **schedulability tests**

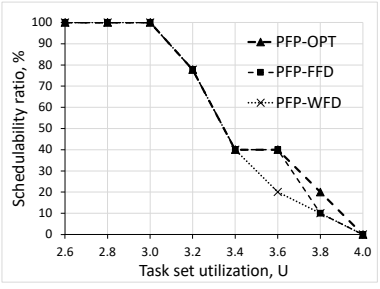
Figure 11: Evaluation results for a varying number of tasks n, for $m = 4$ processors



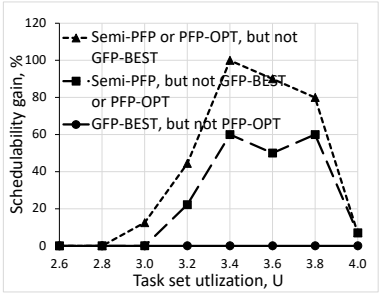
(a) **Schedulability ratios for** GFP, Semi-PFP, **and** PFP **schedulers**



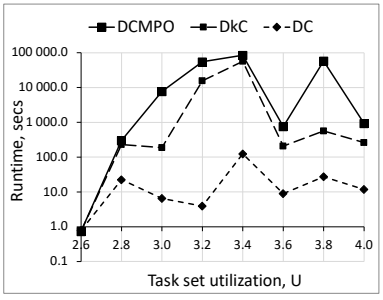
(b) **Schedulability ratios of priority assignment heuristics for** GFP



(c) **Schedulability ratios of partitioning heuristics**

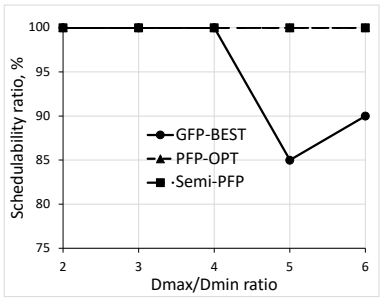


(d) **Schedulability gain of** GFP, Semi-PFP, **and** PFP

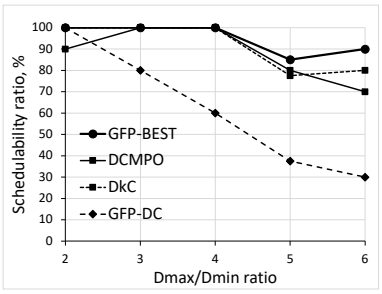


(e) **Runtime of an exact schedulability test for** GFP-DCMPO

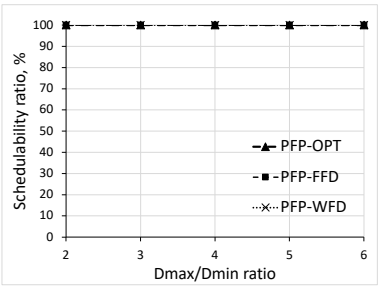
Figure 12: Evaluation results for task set utilization U, for $m = 4$ processors



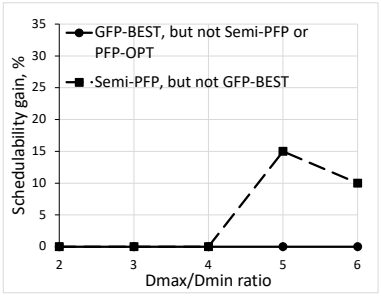
(a) **Schedulability ratios for** GFP, Semi-PFP, **and** PFP **schedulers**



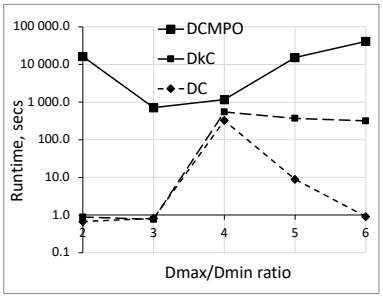
(b) **Schedulability ratios of priority assignment heuristics for** GFP



(c) **Schedulability ratios of partitioning heuristics**

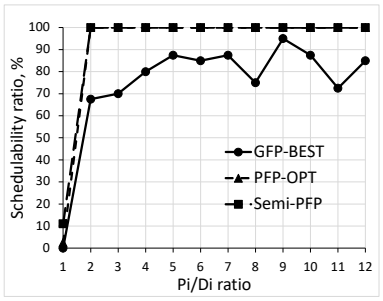


(d) **Schedulability gain of** GFP, Semi-PFP, **and** PFP

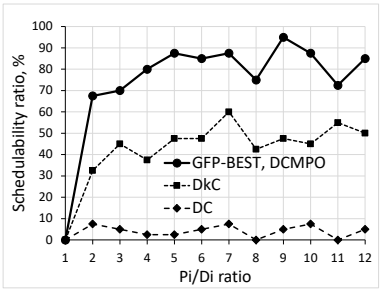


(e) **Runtime of an exact schedulability test for** GFP-DCMPO

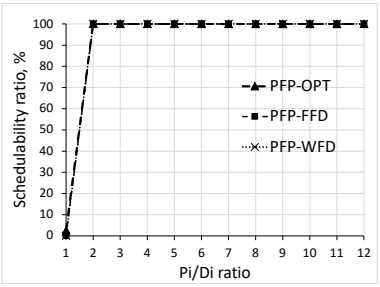
Figure 13: Evaluation results for a varying $D_{\max}/D_{\min}$ ratio, for $m = 4$ processors



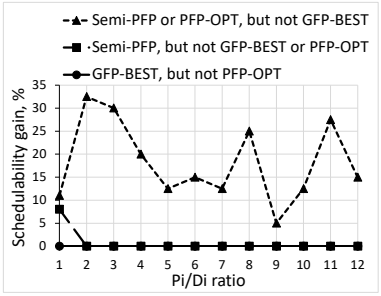
(a) **Schedulability ratios for** GFP, Semi-PFP, **and** PFP **schedulers**



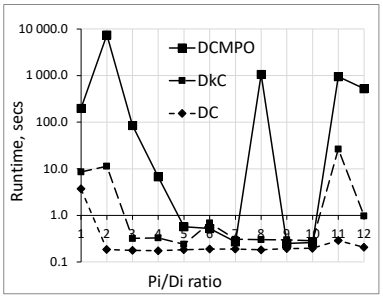
(b) **Schedulability ratios of priority assignment heuristics for** GFP



(c) **Schedulability ratios of partitioning heuristics**



(d) **Schedulability gain of** GFP, Semi-PFP, **and** PFP



(e) **Runtime of an exact schedulability test for** GFP-DCMPO

Figure 14: Evaluation results for a varying ratio of task periods to deadlines P_i/D_i, for $m = 4$ processors

Section 2.3.2. Figs. 5e-9e report the ratio of the number of all feasible priority assignments (that is $n!$ cases for n tasks) to the number of priority assignments actually examined by our test, until some schedulable priority assignment is found, or a task set is confirmed unschedulable.

Finally, for completeness, we evaluate the performance of pruning rules for searching optimal task priorities for GFP, which are formulated in Observations 1 and **??** of Section 2.3.2. The results are reported in Fig. 5e-9e. The reported gain is computed as the ratio of all feasible priority assignments to the number of priority assignments actually examined, until some schedulable priority assignment is found, or a task set is confirmed unschedulable. We observe a notable reduction of the number of cases to be examined over a brute-force search; in certain cases, the gain exceeds $3-5$ times, sometimes yielding the reduction of 10 times. We also report the measured runtime for exact schedulability tests in Figs. 5f-9f, and 10c-14c.

4 CONCLUSION

We evaluated the performance of global, partitioned, and semi-partitioned fixed-priority schedulers. Unlike other works, our evaluation relies on an exact schedulability test for GFP, optimal task partitioning, and optimal priority assignment. We demonstrated the incomparability of GFP, PFP, and Semi-PFP schedulers; the schedulability gain of one approach over another might exceed 40-50%, for certain task set settings. GFP significantly outperforms PFP for task sets with a larger number of heavy tasks, and task sets with a larger ratio of the maximum to the minimum task periods. There are multiple task sets, that are schedulable either by GFP, or Semi-PFP, or PFP, but not all schedulers simultaneously. Due to the exponential computational complexity of the available exact schedulability tests, our evaluation is constrained to task sets with a limited number of tasks with small integer parameters. Still, despite restricted task set settings, our evaluation provides some glimpse on the performance behavior of considered scheduling strategies.

We note that, theoretically, global scheduling generalizes partitioned scheduling. The fact that partitioned FP in some cases outperforms global FP only demonstrates the suboptimality of static task priorities over dynamic ones, rather than the suboptimality of global scheduling itself.

To make this evaluation possible, we also derived the method to compute optimal task priorities for GFP, by combining an exhaustive search with some pruning methods. To motivate the need for optimal task priorities, we demonstrated a poor performance of some suboptimal widely used priority assignment heuristics, including deadline-monotonic assignment. We also confirmed DkC and DCMPO priority assignment heuristics to significantly outperform several other available heuristics, for our experiment settings.

Finally, for partitioned FP scheduling, we analyzed the performance of an optimal task partitioning over FFD and WFD suboptimal heuristics. We observed no significant performance difference, except of a lower performance of WFD for task sets with no or one heavy task.

REFERENCES

[1] Bjorn Anderson and Eduardo Tovar. 2006. Multiprocessor scheduling with few preemptions. In *RTCSA*.

[2] James H. Anderson, Vasile Bud, and UmaMaheswari C. Devi. 2005. An EDF-based Scheduling Algorithm for Multiprocessor Soft Real-Time Systems. In *17th Euromicro Conference on Real-Time Systems (ECRTS'05)*.

[3] Neil C. Audsley, Alan Burns, Robert I. Davis, Ken W. Tindell, and Andy J. Wellings. 1995. Fixed Priority Pre-emptive Scheduling: An Historical Perspective. *Real-Time Systems Journal* (1995).

[4] Theodor Baker and Sanjoy Baruah. 2007. *Handbook of Real-Time and Embedded Systems.* CRC Press, Chapter 2. Schedulability analysis of multiprocessor sporadic task systems.

[5] Theodore P. Baker. 2005. A Comparison of Global and Partitioned EDF Schedulability Tests for Multiprocessors. In *Proceedings of Real-Time and Network Systems (RTNS)*.

[6] Sanjoy Baruah. 2007. Techniques for multiprocessor global schedulability analysis. In *Proceedings of the IEEE Real-Time Systems Symposium*.

[7] Sanjoy Baruah and Enrico Bini. 2008. Partitioned scheduling of sporadic task systems: an ILP-based approach. In *DASIP*.

[8] Sanjoy Baruah and Alan Burns. 2006. Sustainable scheduling analysis. In *Proceedings of the IEEE Real-Time Systems Symposium*.

[9] Sanjoy Baruah and Nathan Fisher. 2005. The partitioned multiprocessor scheduling of sporadic task systems. In *RTSS*.

[10] S. K. Baruah, N. K. Cohen, C. G. Plaxton, and D. A. Varvel. 1996. Proportionate progress: A notion of fairness in resource allocation. *Algorithmica* 15, 600 (1996).

[11] Marko Bertogna, Michele Cirinei, and Giuseppe Lipari. 2009. Schedulability analysis of global scheduling algorithms on multiprocessor platforms. *IEEE Transactions on Parallel and Distributed Systems* 20, 4 (April 2009), 553–566.

[12] Enrico Bini and Giorgio Buttazzo. 2005. Measuring the performance of schedulability tests. *Real-Time Systems Journal* 30, 1-2 (2005), 129–154.

[13] Alessandro Biondi and Youcheng Sun. 2018. On the ineffectiveness of 1/m-based interference bounds in the analysis of global EDF and FIFO scheduling. *Real-Time Systems* (2018).

[14] Vincenzo Bonifaci and Alberto Marchetti-Spaccamela. 2012. Feasibility Analysis of Sporadic Real-Time Multiprocessor Task Systems. *Algorithmica* (2012).

[15] Björn B. Brandenburg and Mahircan Gül. 2016. Global Scheduling Not Required: Simple, Near-Optimal Multiprocessor Real-Time Scheduling with Semi-Partitioned Reservations. In *Proceedings of Real-Time Systems Symposium (RTSS)*. IEEE.

[16] Richard L. Burden, J. Douglas Faires, and Annette M. Burden. 2015. *Numerical Analysis* (9th ed.). Cengage Learning, Chapter 2.2 Fixed-Point Iteration.

[17] Artem Burmyakov. 2016. *Schedulability Analysis of Multiprocessor Real-time Systems Using Pruning.* Ph.D. Dissertation. University of Porto, Faculty of Engineering.

[18] Artem Burmyakov, Enrico Bini, and Eduardo Tovar. 2015. An exact schedulability test for global FP using state space pruning. In *Proceedings of the 23rd International Conference on Real Time and Networks Systems (RTNS2015)*. 225–234.

[19] Robert Davis and Alan Burns. 2009. Priority assignment for global fixed priority pre-emptive scheduling in multiprocessor real-time systems. In *Proceedings of the 30th IEEE Real-Time Systems Symposium (RTSS 2009)*. 398–409.

[20] Robert Davis and Alan Burns. 2011. Improved priority assignment for global fixed priority pre-emptive scheduling in multiprocessor real-time systems. *Real-Time Systems* 47, 1 (2011), 1–40.

[21] S. K. Dhall and C. L. Liu. 1978. On a real-time scheduling problem. *Operations Research* 26, 1 (1978), 127–140.

[22] R.F. Freund, M. Gherrity, S. Ambrosius, M. Campbell, M. Halderman, D. Hensgen, E. Keith, T. Kidd, M. Kussow, J.D. Lima, F. Mirabile, L. Moore, B. Rust, and H.J. Siegel. 1998. Scheduling resources in multi-user, heterogeneous, computing environments with SmartNet. In *Proceedings Seventh Heterogeneous Computing Workshop (HCW'98)*. 184–199. https://doi.org/10.1109/HCW.1998.666558

[23] Gilles Geeraerts, Joël Goossens, and Markus Lindström. 2013. Multiprocessor schedulability of arbitrary-deadline sporadic tasks: complexity and antichain algorithm. *Real-Time Systems* (2013).

[24] Giovani Gracioli, Antonio Augusto Frohlich, Rodolfo Pellizzoni, and Sebastian Fischmeinster. 2013. Implementation and evaluation of global and partitioned scheduling in a real-time OS. *Real-Time Systems* 49, 6 (November 2013), 669–714.

[25] Nan Guan, Martin Stigge, Wang Yi, and Ge Yu. 2009. New Response Time Bounds for Fixed Priority Multiprocessor Scheduling. In *RTSS*.

[26] David Johnson. 1973. Near-optimal bin packing algorithms. *Thesis (Ph. D.)–Massachusetts Institute of Technology, Dept. of Mathematics* (1973).

[27] Mathai Joseph and Paritosh K. Pandya. 1986. Finding Response Times in a Real-Time System. *Comput. J.* 29, 5 (Oct. 1986), 390–395.

[28] Shinpei Kato and Nobuyuki Yamasaki. 2009. Semi-partitioned Fixed-Priority Scheduling on Multiprocessors. In *2009 15th IEEE Real-Time and Embedded Technology and Applications Symposium*.

[29] Sylvain Lauzac, Rami Melhem, and Daniel Mosse. 1998. Comparison of global and partitioning schemes for scheduling rate monotonic tasks on a multiprocessor.

In *Proceedings of 10th EUROMICRO Workshop on Real-Time Systems*.

[30] Joseph Leung and Jennifer Whitehead. 1982. On the complexity of fixed-priority scheduling of periodic, real-time tasks. *Performance Evaluation* 2, 4 (December 1982), 237–250.

[31] C. Liu and J. Layland. 1973. Scheduling algorithms for multiprogramming in a hard real-time environment. *Journal of the ACM (JACM)* 20 (January 1973).

[32] Sanjay Moulik, Zinea Das, Rajesh Devaraj, and Shounak Chakraborty. 2021. SEAMERS: A Semi-partitioned Energy-Aware scheduler for heterogeneous MulticorE Real-time Systems. *Journal of Systems Architecture* (2021).

[33] Sanjay Moulik, Arnab Sarkar, and Hemangee K.Kapoor. 2018. Energy aware frame based fair scheduling. *Sustainable Computing: Informatics and Systems* (2018).

[34] Sanjay Moulik, Arnab Sarkar, and Hemangee K.Kapoor. 2021. TARTS: A Temperature-Aware Real-Time Deadline-Partitioned Fair Scheduler. *Journal of Systems Architecture* (2021).

[35] Youcheng Sun and Giuseppe Lipari. 2014. A Weak Simulation Relation for Real-Time Schedulability Analysis of Global Fixed Priority Scheduling Using Linear Hybrid Automata. In *Proceedings of the 22nd International Conference on Real-Time Networks and Systems*.

[36] Youcheng Sun and Marco Di Natale. 2017. *Assessing the Pessimism of Current Multicore Global Fixed-Priority Schedulability Analysis.* Technical Report. University of Oxford.

[37] Junlong Zhou, Jianming Yan, Jing Chen, and Tongquan Wei. 2016. Peak Temperature Minimization via Task Allocation and Splitting for Heterogeneous MPSoC Real-Time Systems. *J. Signal Process. Syst.* (2016).